AZURE LANDING ZONE

Interview Deep-Dive Notes

SECTION 10 of 10 | DEVSECOPS

Concept Notes + Interview Q&A • 12 Topics

Prepared for: Ajeet Singh — DevOps / DevSecOps Engineer

SECTION 10 — DEVSECOPS

DevSecOps is the discipline of shifting security left -- embedding automated scanning, policy enforcement, and compliance checks directly into the infrastructure-as-code and CI/CD pipeline, instead of treating security as a manual gate applied only after deployment. Interviewers probe this to see if you understand where each tool fits in the pipeline and how findings translate into actual gates that block or allow a release.

---

1. tfsec

Concept:

tfsec is a static analysis security scanner purpose-built for Terraform code. It parses HCL (without needing to run terraform plan or apply) and flags misconfigurations against a built-in rule set -- for example, a storage account with public access enabled, or a network security group allowing inbound traffic from any source. It is typically run as a pre-commit hook or a CI pipeline step immediately after a pull request is opened, catching insecure infrastructure before it is ever provisioned.

Q1: Why run tfsec against Terraform code instead of waiting to scan the deployed Azure resources afterward?

Scanning the code itself catches misconfigurations before any cloud resource is created, which is far cheaper and faster to fix than remediating an already-deployed resource -- this is the core 'shift-left' principle: the earlier a security issue is caught in the pipeline, the lower the cost and risk of fixing it.

Q2: How do you handle a tfsec finding that is a deliberate, approved exception rather than a real issue?

tfsec supports inline suppression comments (e.g. #tfsec:ignore:) directly in the Terraform file, ideally with a comment explaining why the exception is acceptable, so the suppression is visible in code review and tracked in version control rather than silently bypassed.

Q3: Where does tfsec typically sit in a CI/CD pipeline relative to Checkov or Terratest?

tfsec usually runs as an early, fast static check right after code is committed or a PR is opened -- often alongside or as an alternative to Checkov for policy-style checks -- while Terratest runs later, after a plan or apply, since it validates actual provisioned infrastructure behavior rather than just the static code.

2. Checkov

Concept:

Checkov is a static-analysis policy scanner similar to tfsec but broader in scope -- it supports Terraform, CloudFormation, Kubernetes manifests, Dockerfiles, ARM/Bicep, and Serverless configurations in a single tool, checking them against thousands of built-in policies as well as custom policies written in Python or YAML. It is commonly chosen when a team needs one consistent scanning tool across multiple IaC languages rather than a separate tool per format.

Q1: When would a team choose Checkov over tfsec if both can scan Terraform?

When the organization uses more than one IaC format -- for example Terraform for some workloads and Kubernetes manifests or Dockerfiles for others -- Checkov's multi-framework support means one tool and one policy set can cover all of them, reducing tooling sprawl compared to running a separate scanner per format.

Q2: How can Checkov be extended with organization-specific rules beyond its built-in policy set?

Checkov supports custom policies written either in Python (using its policy SDK) or in a structured YAML format, letting a platform team encode internal standards -- such as a mandatory tagging scheme or an approved VM SKU list -- as scannable rules alongside the built-in checks.

Q3: What is a typical failure mode when Checkov is only run locally by developers and not enforced in CI?

Findings become optional and inconsistently addressed -- a developer might fix issues locally but another might skip the scan entirely, so misconfigurations still reach production; Checkov needs to be wired into the CI/CD pipeline as a required, blocking step for the checks to function as an actual security gate rather than a suggestion.

3. Terratest

Concept:

Terratest is a Go-based testing framework for validating that infrastructure code actually behaves as intended once deployed -- unlike tfsec/Checkov, which analyze static code, Terratest typically provisions real (often short-lived, disposable) infrastructure in a test environment, runs assertions against it (e.g., an HTTP endpoint returns 200, a VM is reachable on the expected port), and then tears it down.

Q1: How does Terratest's approach differ fundamentally from tfsec or Checkov?

tfsec and Checkov are static analysis tools -- they inspect the Terraform code itself without deploying anything. Terratest is a functional/integration testing tool -- it actually applies the Terraform configuration to real infrastructure (usually in an isolated test subscription) and validates the live, running behavior, then destroys the test resources afterward.

Q2: Why is Terratest usually run against a disposable test environment rather than production?

Because it provisions real cloud resources to validate behavior, running it against production would risk unintended side effects or cost; a disposable test subscription or resource group lets the test suite create, validate, and destroy infrastructure repeatedly and safely as part of CI without touching real workloads.

Q3: What kind of issues can Terratest catch that a static scanner like tfsec cannot?

Runtime and integration issues -- for example, whether two resources actually communicate correctly once deployed (like an App Service reaching a Key Vault through a private endpoint), or whether an output value is genuinely usable -- since these depend on real provisioned state, not just the syntax or declared configuration of the code.

4. Trivy

Concept:

Trivy is an open-source, all-in-one scanner that detects vulnerabilities, misconfigurations, secrets, and license issues across container images, filesystems, Git repositories, and Infrastructure-as-Code files. In a Landing Zone pipeline it is most commonly used to scan container images for known CVEs before they are pushed to a registry or deployed to AKS.

Q1: What is the difference between Trivy and a tool like tfsec, given both can scan Infrastructure-as-Code?

Trivy is broader in scope -- beyond IaC misconfiguration scanning, it also scans container images for OS and application-level vulnerabilities (CVEs), detects hardcoded secrets, and checks license compliance, whereas tfsec is focused specifically on Terraform misconfigurations.

Q2: At what stage of the CI/CD pipeline is Trivy typically used to scan a container image, and why there?

Immediately after the image is built but before it is pushed to the container registry (e.g., Azure Container Registry) -- scanning at this point catches vulnerable base images or dependencies before the image becomes available for deployment, preventing a vulnerable image from ever reaching AKS.

Q3: How would you configure Trivy to fail a pipeline only on high-severity vulnerabilities while still reporting lower-severity findings?

Trivy supports a severity filter (e.g., --severity HIGH,CRITICAL combined with --exit-code 1) so the scan can be configured to fail the build only when high or critical vulnerabilities are found, while lower-severity findings are still reported in the output for visibility without blocking the pipeline.

5. Microsoft Defender for Cloud

Concept:

Microsoft Defender for Cloud is Azure's native Cloud Security Posture Management (CSPM) and Cloud Workload Protection Platform (CWPP), giving a Secure Score across subscriptions, continuous posture recommendations, and workload-specific threat protection plans (for servers, containers, SQL, Key Vault, storage, and more). In a Landing Zone it is typically enabled centrally at the Management Group level so every subscription is assessed consistently.

Q1: What is the difference between Defender for Cloud's free CSPM tier and its paid Defender plans?

The free tier provides baseline posture recommendations and the overall Secure Score across resources. The paid Defender plans (e.g., Defender for Servers, Defender for Containers, Defender for SQL) add active threat detection, just-in-time VM access, vulnerability assessment, and workload-specific runtime protection -- each plan billed and enabled individually per resource type.

Q2: How does Defender for Cloud complement Azure Policy in a Landing Zone rather than duplicate it?

Azure Policy enforces and blocks configuration at deployment time (preventive), while Defender for Cloud continuously assesses posture and detects active threats after resources exist (detective and responsive) -- together they cover the full lifecycle: Policy stops misconfigurations before they happen, Defender for Cloud finds ones that slip through or emerge from runtime threats.

Q3: Where is Defender for Cloud typically enabled and centrally monitored in an Enterprise-Scale Landing Zone?

It is enabled at the Management Group level (often via the Management subscription's Log Analytics workspace as the central export destination) so Secure Score and recommendations roll up consistently across every landing zone subscription, giving the platform/security team one unified dashboard instead of checking each subscription individually.

6. Azure Policy as Code

Concept:

Policy as Code is the practice of authoring, testing, versioning, and deploying Azure Policy definitions, initiatives, and assignments through the same CI/CD and source-control workflow used for application code -- typically via Terraform (azurerm_policy_definition, azurerm_policy_assignment) or Bicep/ARM templates -- rather than clicking through the Azure Portal.

Q1: Why manage Azure Policy through Terraform/Bicep instead of assigning policies manually in the Portal?

Manual Portal assignment is not version-controlled, not peer-reviewed, and not repeatable across environments -- Policy as Code gives a Git history of every governance change, allows pull-request review before a new Deny policy goes live, and lets the exact same policy set be redeployed consistently to a new Management Group or environment.

Q2: How do you safely test a new policy definition before it is enforced across production landing zones when managing policy as code?

Deploy the assignment first with enforcementMode set to DisabledDoNotEnforce (the same setting used for manual rollouts), run it through a lower environment or a subset of subscriptions, review the compliance results, and only merge the change to set enforcement to Enabled once the impact is validated -- following the same CI/CD promotion pattern as any other infrastructure change.

Q3: What CI/CD safeguard is commonly added specifically for pipelines that deploy Policy definitions with a Deny effect?

A mandatory manual approval/review gate before applying to production Management Groups, since a mistaken Deny policy can block deployments across every subscription beneath that scope -- unlike most infrastructure changes, a bad policy assignment can silently halt unrelated teams' pipelines tenant-wide.

7. Secret Scanning

Concept:

Secret Scanning is the automated detection of hardcoded credentials, API keys, connection strings, or certificates accidentally committed into source code, using tools such as GitHub Advanced Security secret scanning, GitLeaks, or TruffleHog. It is typically run both as a pre-commit hook (to stop a secret before it is ever pushed) and as a CI pipeline step (to catch anything that slips through, including in historical commits).

Q1: Why is scanning only the latest commit for secrets insufficient, and what else needs to be checked?

A secret can be committed and then 'removed' in a later commit, but it still exists permanently in Git history and can be retrieved by anyone with repository access -- so effective secret scanning also needs to scan full commit history (tools like GitLeaks and TruffleHog support this), and a leaked secret found in history must be rotated, not just deleted from the latest commit.

Q2: What is the correct remediation step once a real secret is found committed to a repository, beyond removing it from the code?

The exposed credential must be rotated/revoked immediately, since it should be treated as compromised the moment it was pushed -- simply removing it from the latest commit or even rewriting Git history does not undo any unauthorized use that may have already occurred with the old value.

Q3: How does secret scanning fit alongside Azure Key Vault in a secure pipeline design?

Secret scanning is a detective control catching accidental leaks of credentials that should never have been hardcoded in the first place, while Key Vault is the preventive design pattern -- storing secrets centrally and having applications/pipelines retrieve them at runtime via managed identity, so there is ideally nothing sensitive in source code for the scanner to ever find.

8. SAST (Static Application Security Testing)

Concept:

SAST analyzes application source code, bytecode, or binaries without executing the program, looking for security vulnerabilities such as SQL injection, cross-site scripting, insecure deserialization, or hardcoded credentials directly in the code logic. It runs early in the pipeline (on every commit or pull request), giving developers fast feedback before the application is even built or deployed.

Q1: What is the key difference between SAST and DAST in terms of what each can and cannot find?

SAST examines source code directly without running the application, so it can pinpoint the exact vulnerable line of code but may produce false positives since it lacks runtime context; DAST tests a running application from the outside like an attacker would, catching real, exploitable runtime issues but without visibility into which line of code caused them, and only after the application is deployed and running.

Q2: Why does SAST typically produce a higher rate of false positives than DAST?

Because SAST analyzes code patterns without actually executing the application, it flags code that looks potentially vulnerable based on static patterns even when runtime conditions (such as input sanitization happening elsewhere, or the vulnerable code path being unreachable in practice) mean it is not actually exploitable, requiring a human to triage and confirm real findings.

Q3: At what point in a CI/CD pipeline is SAST typically integrated, and why there?

SAST is integrated as early as possible -- ideally on every pull request before merge -- since it only needs source code and not a running application, giving developers immediate, low-cost feedback on vulnerabilities while the code is still fresh in review, well before the later build, deploy, or DAST stages.

9. DAST (Dynamic Application Security Testing)

Concept:

DAST tests a running, deployed application from the outside -- typically in a staging or pre-production environment -- by sending crafted requests (SQL injection attempts, malformed input, authentication bypass attempts) and observing the application's actual responses, without any access to source code. Tools like OWASP ZAP or Burp Suite are commonly automated into a pipeline stage that runs after deployment to a test environment.

Q1: Why must DAST run against a deployed, running application rather than against source code?

DAST simulates real attacker behavior by sending live HTTP requests to a running application and observing its actual behavior and responses -- it has no access to source code and cannot analyze logic statically, so it inherently requires a live, reachable instance of the application to test against.

Q2: What is a practical limitation of running DAST scans in a CI/CD pipeline compared to SAST?

DAST scans are significantly slower, since they require the application to be fully built, deployed to an environment, and then crawled/tested with live requests, whereas SAST can analyze source code directly in seconds to minutes -- this typically means DAST runs less frequently (e.g., nightly or per release) rather than on every single pull request.

Q3: How do SAST and DAST complement each other rather than one replacing the other?

SAST catches vulnerabilities early in the code itself with precise line-level detail but can miss issues that only manifest at runtime (such as misconfigured authentication or environment-specific issues), while DAST catches real, exploitable runtime behavior an attacker could actually trigger but only after deployment and without pinpointing the exact code location -- using both together covers a broader range of vulnerability types across the pipeline.

10. Dependency Scanning

Concept:

Dependency Scanning (Software Composition Analysis) checks an application's third-party libraries and packages -- npm, NuGet, pip, Maven, etc. -- against known vulnerability databases (such as the National Vulnerability Database) to flag CVEs in dependencies the team did not write themselves. Tools include GitHub Dependabot, Snyk, and OWASP Dependency-Check, usually run on every commit/PR and on a recurring schedule since new CVEs are published continuously.

Q1: Why is dependency scanning needed even when an application's own code has already passed a SAST scan?

SAST only analyzes code the team actually wrote; the vast majority of a modern application's code is often third-party libraries pulled in as dependencies, and a vulnerability in one of those libraries (e.g., a widely used logging library) is completely invisible to SAST -- dependency scanning specifically targets this separate risk surface.

Q2: Why does dependency scanning need to run on a recurring schedule, not just once at commit time?

New CVEs are discovered and published continuously even in dependencies that have not changed in the application's own code -- a library considered safe last month might have a newly disclosed vulnerability today, so scans need to re-run periodically (e.g., via Dependabot's scheduled checks) to catch newly

disclosed issues in already-deployed dependencies.

Q3: What is a common automated remediation pattern paired with dependency scanning tools like Dependabot?

Automatic pull request generation -- when a vulnerable dependency version is detected, the tool opens a PR bumping it to the patched version automatically, letting the team simply review and merge the fix rather than manually tracking down and updating the affected package.

11. Container Image Scanning

Concept:

Container Image Scanning inspects a built container image layer by layer -- the base OS image, installed packages, and application dependencies -- for known vulnerabilities, misconfigurations, and embedded secrets, using tools such as Trivy, Microsoft Defender for Containers, or Grype. It is run both at build/push time in the CI pipeline and continuously against images already stored in a registry like Azure Container Registry, since new CVEs can be published after an image was pushed.

Q1: Why scan a container image again after it is already sitting in Azure Container Registry, if it already passed a scan at build time?

The scan at build time only reflects vulnerabilities known at that moment -- new CVEs are disclosed continuously, so an image considered clean when pushed can later be found to contain a newly discovered vulnerability in its base OS or packages; continuous registry scanning (e.g., via Defender for Containers) re-evaluates stored images against the latest vulnerability data.

Q2: What is the difference between scanning the Dockerfile/build instructions versus scanning the final built image?

Scanning the Dockerfile (often via a tool like Checkov or Trivy's config scan) checks for misconfigurations in how the image is built, such as running as root or exposing unnecessary ports, while scanning the final built image inspects the actual resulting layers -- the real base OS version and installed package versions -- for known CVEs, which the Dockerfile alone cannot reveal.

Q3: How should a CI/CD pipeline gate a release based on container image scan severity?

Typically configured so the pipeline fails the build and blocks the push to the registry (or blocks deployment to AKS) when critical or high-severity vulnerabilities with an available fix are found, while lower-severity or currently unfixable findings are logged and tracked without blocking, avoiding an unworkable pipeline that never ships.

12. CI/CD Security Gates

Concept:

A Security Gate is a defined checkpoint in the CI/CD pipeline where the build or release is automatically blocked from proceeding unless specific security conditions are met -- such as a clean SAST scan, no critical container vulnerabilities, no secrets detected, and a passing Policy-as-Code compliance check. This is what turns individual scanning tools (tfsec, Trivy, SAST, DAST, dependency scanning) from advisory reports into actual enforced controls.

Q1: What is the practical difference between a scanning tool producing a report and that same tool acting as a security gate?

A report alone is only useful if someone actively reads and acts on it, which is inconsistent at scale; a gate makes the pipeline itself fail and block the merge, build, or deployment automatically when the scan result violates a defined threshold, removing reliance on a human remembering to check the report.

Q2: How would you design security gates across a pipeline to avoid blocking every single build on minor findings?

Tier the gates by severity and pipeline stage -- for example, fail fast on critical/high-severity findings with a known fix (secrets, critical CVEs, Deny-level policy violations), while logging medium/low findings as warnings tracked in a backlog rather than blocking, so the gate stops genuinely dangerous releases without making the pipeline unusable.

Q3: In an Enterprise-Scale Landing Zone, which security gates are typically non-negotiable (hard fail) versus advisory?

Hard-fail gates typically include secret detection (any real secret found), Azure Policy Deny-level violations, and critical/high CVEs with an available patch in container images or dependencies; advisory gates typically include SAST/DAST medium-or-lower findings and Advisor-style cost or best-practice recommendations, which are tracked but do not block the release.